Informe de laboratorio 09

Tema: Métodos Heurísticos: Heurísticas Constructivas y de Búsqueda Local.

Nota

Estudiante	Escuela	Asignatura
Capatinta Almanza Camilo B. A. ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Laboratorio	Tema	Duración
09	Métodos Heurísticos: Heurísticas Constructivas y de Búsqueda Local.	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	——	——

1. Ejercicios propuestos por el docente

1.1. Construcción de una ruta con Nearest Neighbor

Construir una ruta para el TSP de cinco ciudades del Laboratorio 8 usando la misma matriz de distancias. Aplicar Nearest Neighbor empezando en la ciudad 0 y comparar el costo obtenido con el óptimo conocido, que es 70.

```cpp
#include <iostream>
#include <vector>

using namespace std;

const int N = 5;

int dist[N][N] = {
    { 0, 10, 15, 20, 25},
    {10,  0, 35, 25, 30},
    {15, 35,  0, 30,  5},
    {20, 25, 30,  0, 15},
    {25, 30,  5, 15,  0}
```

```cpp
};

int main() {
    int inicio = 0;

    vector<bool> visitado(N, false);
    vector<int> ruta = { inicio };
    visitado[inicio] = true;

    for (int paso = 1; paso < N; paso++) {
        int actual = ruta.back();
        int masCercano = -1;
        int mejorDist = 1000000;

        for (int c = 0; c < N; c++) {
            if (!visitado[c] && dist[actual][c] < mejorDist) {
                mejorDist = dist[actual][c];
                masCercano = c;
            }
        }

        ruta.push_back(masCercano);
        visitado[masCercano] = true;
    }

    int costo = 0;
    for (int i = 0; i + 1 < N; i++) costo += dist[ruta[i]][ruta[i + 1]];
    costo += dist[ruta.back()][ruta[0]];

    cout << "Ruta (Nearest Neighbor): ";
    for (int c : ruta) cout << c << " -> ";
    cout << ruta[0] << endl;
    cout << "Costo total: " << costo << endl;

    return 0;
}
```

```
camiloasd …/Unidad3/lab09/e-p  main !?  11:28  g++ tsp_nearest_neighbor.cpp -o tsp_nearest_neighbor && ./tsp_nearest_neighbor
Ruta (Nearest Neighbor): 0 -> 1 -> 3 -> 4 -> 2 -> 0
Costo total: 70

camiloasd …/Unidad3/lab09/e-p  main !?  11:28
```

Figura 1: Ruta para el TSP de 5 ciudades.

La heurística comienza en la ciudad 0 y, en cada paso, elige la ciudad no visitada más cercana a la ciudad actual. Cuando todas las ciudades han sido visitadas, cierra el recorrido regresando al origen. El programa almacena las distancias en una matriz y usa un vector para marcar las ciudades visitadas y otro para construir la ruta. Para esta instancia obtiene la ruta 0–1–3–4–2–0, de costo 70.

1.2. Mejora de una ruta con búsqueda local 2-opt

Aplicar 2-opt a la ruta inicial 0–1–2–4–3–0, obtenida con Nearest Neighbor al empezar en la ciudad 2, cuyo costo es 85. Comparar la ruta mejorada con el óptimo conocido del TSP, de costo 70.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

const int N = 5;

int dist[N][N] = {
    { 0, 10, 15, 20, 25},
    {10,  0, 35, 25, 30},
    {15, 35,  0, 30,  5},
    {20, 25, 30,  0, 15},
    {25, 30,  5, 15,  0}
};

int costoRuta(const vector<int>& ruta) {
    int c = 0;

    for (int i = 0; i + 1 < N; i++) c += dist[ruta[i]][ruta[i + 1]];
    c += dist[ruta[N - 1]][ruta[0]];

    return c;
}

int main() {
    vector<int> ruta = { 0, 1, 2, 4, 3 };

    cout << "Ruta inicial: ";
    for (int c : ruta) cout << c << " -> ";
    cout << ruta[0] << "  (costo " << costoRuta(ruta) << ")" << endl;

    bool mejora = true;
    while (mejora) {
        mejora = false;

        for (int i = 1; i < N - 1; i++) {
            for (int j = i + 1; j < N; j++) {
                vector<int> nueva = ruta;
                reverse(nueva.begin() + i, nueva.begin() + j + 1);

                if (costoRuta(nueva) < costoRuta(ruta)) {
                    ruta = nueva;
                    mejora = true;
                }
            }
        }
    }

    cout << "Ruta (2-opt): ";
    for (int c : ruta) cout << c << " -> ";
    cout << ruta[0] << endl;
    cout << "Costo total: " << costoRuta(ruta) << endl;

    return 0;
```

```
}
```

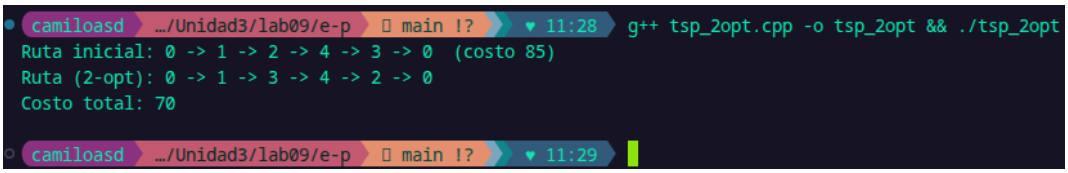


Figura 2: Ruta mejorada con 2-opt.

El algoritmo parte de una ruta completa y prueba invertir segmentos entre distintas posiciones. Calcula el costo de cada ruta candidata y acepta una inversión solo cuando reduce estrictamente el costo. Repite las pruebas hasta que ninguna inversión produzca una mejora. En este ejemplo invierte un segmento y obtiene la ruta 0–1–3–4–2–0, reduciendo el costo de 85 a 70.

2. Ejercicios/Problemas propuestos

2.1. Ejercicio 1

Implementa Nearest Neighbor para un TSP de 15 ciudades generadas aleatoriamente, y ejecútalo partiendo de cada una de las 15 ciudades como origen. ¿El resultado cambia según el punto de partida?

```cpp
#include "tsp.h"

int main() {
    const int N = 15;
    Instancia I = generar(N);

    vector<double> costos(N);
    printf("%-7s %10s\n", "Origen", "Costo NN");
    for (int o = 0; o < N; o++) {
        costos[o] = costo(vecinoCercano(o, I.d), I.d);
        printf("%-7d %10.3f\n", o, costos[o]);
    }

    Resumen r = resumir(costos);
    printf("\nMejor: %.3f (origen %d) | Peor: %.3f | Media: %.3f\n", r.mejor, r.idxMejor,
    ↪  r.peor, r.media);
    printf("Orgenes que alcanzan el mejor: %d/%d | Costos distintos: %d\n", r.enMejor, N,
    ↪  r.distintos);
    printf("¿Cambia según el origen? %s (peor excede al mejor en %.2f%%)\n",
           r.distintos > 1 ? "SÍ" : "NO", 100 * (r.peor - r.mejor) / r.mejor);
}
```

- /relleno

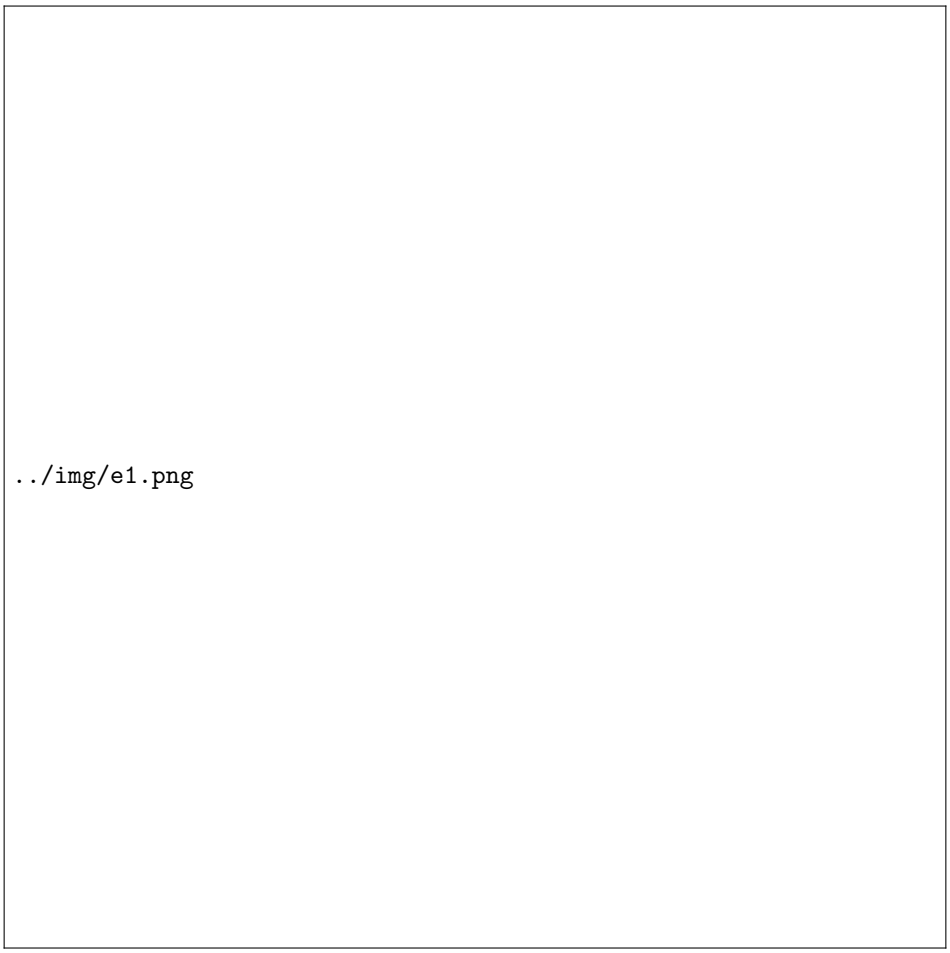


Figura 3: Ejecución del ejercicio 1

2.2. Ejercicio 2

Aplica 2-opt sobre las 15 soluciones del ejercicio anterior. ¿El mejor resultado final sigue dependiendo de la ciudad de origen usada por Nearest Neighbor?

```cpp
// Ejercicio 2: 2-opt sobre las 15 soluciones de Nearest Neighbor (Ejercicio 1).
// Pregunta: ¿el mejor resultado final sigue dependiendo de la ciudad de origen?
#include "tsp.h"

int main() {
    const int N = 15;
    Instancia I = generar(N);  // misma semilla que el Ejercicio 1: mismas ciudades

    vector<double> antes(N), despues(N);
    printf("%-7s %10s %12s %9s\n", "Origen", "NN", "NN + 2-opt", "Mejora");
    for (int o = 0; o < N; o++) {
        Ruta nn = vecinoCercano(o, I.d);
        antes[o] = costo(nn, I.d);
        despues[o] = costo(dosOpt(nn, I.d), I.d);
```

```
15          printf("%-7d %10.3f %12.3f %8.2f%%\n", o, antes[o], despues[o], 100 * (antes[o] -
            ↪  despues[o]) / antes[o]);
16      }
17
18      Resumen a = resumir(antes), d = resumir(despues);
19      printf("\n%-11s %9s %9s %9s %12s %10s\n", "", "Mejor", "Peor", "Media", "Casos mejor",
        ↪  "Distintos");
20      printf("%-11s %9.3f %9.3f %9.3f %9d/%d %10d\n", "NN", a.mejor, a.peor, a.media,
        ↪  a.enMejor, N, a.distintos);
21      printf("%-11s %9.3f %9.3f %9.3f %9d/%d %10d\n", "NN + 2-opt", d.mejor, d.peor, d.media,
        ↪  d.enMejor, N, d.distintos);
22
23      printf("\nDispersión (peor vs mejor): NN %.2f%% -> NN + 2-opt %.2f%%\n",
24             100 * (a.peor - a.mejor) / a.mejor, 100 * (d.peor - d.mejor) / d.mejor);
25      printf("Origen del mejor: NN = %d, NN + 2-opt = %d\n", a.idxMejor, d.idxMejor);
26      printf("El resultado final depende del origen? %s (%d costos distintos; el mejor se
        ↪  alcanza desde %d de %d orgenes)\n",
27             d.distintos > 1 ? "SÍ" : "NO", d.distintos, d.enMejor, N);
28  }
```

- /relleno

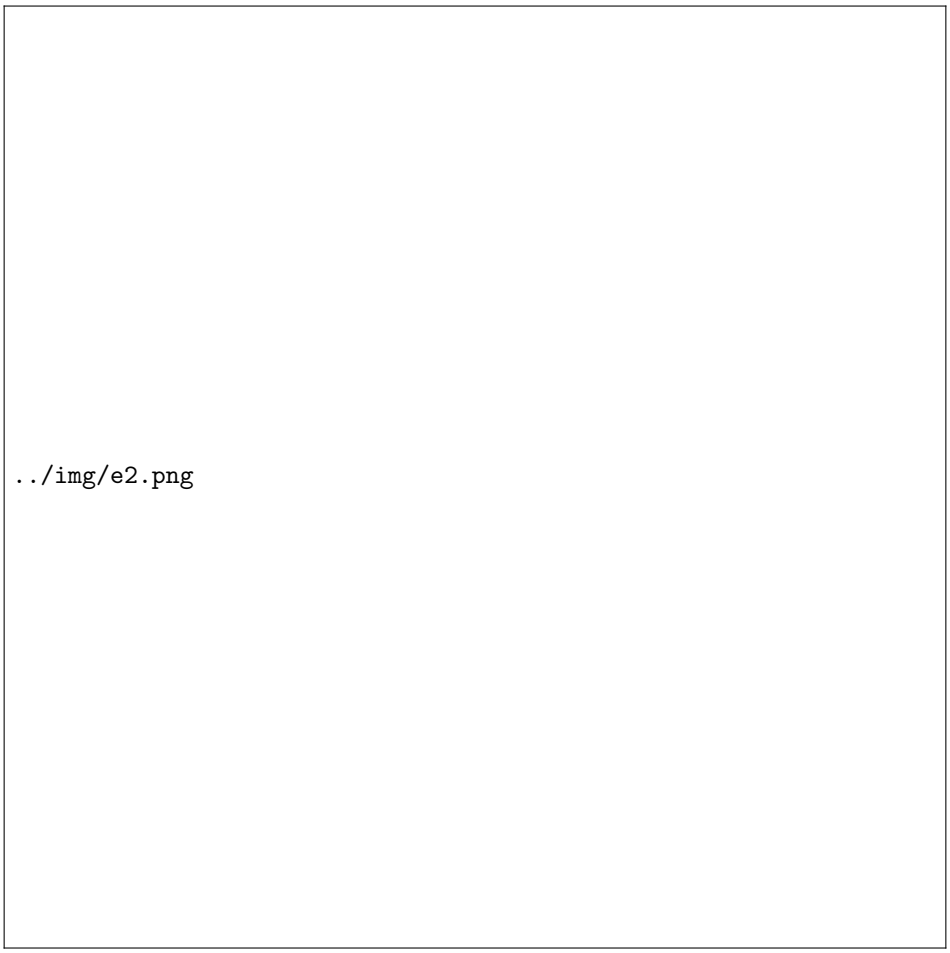


Figura 4: Ejecución del ejercicio 2

2.3. Ejercicio 3

Implementa otra heurística constructiva: la heurística del "vecino más barato"(cheapest insertion), que en cada paso inserta la ciudad no visitada que menos aumenta el costo total, en cualquier posición de la ruta. Compárala contra Nearest Neighbor.

```cpp
// Ejercicio 3: Inserción más barata (cheapest insertion) vs Nearest Neighbor.
// Ambas se ejecutan desde cada origen sobre las mismas 15 ciudades de los ejercicios 1 y 2.
#include "tsp.h"

// Tiempo medio (ms) de ejecutar la heurstica desde los N orgenes; se repite porque una sola corrida dura casi 0.
template <class F> double tiempoMs(F heuristica, const Mat& d, int reps = 2000) {
    auto t0 = chrono::steady_clock::now();
    double sumidero = 0;  // evita que el compilador elimine las llamadas
    for (int k = 0; k < reps; k++)
        for (int o = 0; o < (int)d.size(); o++) sumidero += heuristica(o, d)[0];
    auto t1 = chrono::steady_clock::now();
    return chrono::duration<double, milli>(t1 - t0).count() / reps + 0 * sumidero;
}
```

```cpp

int main() {
    const int N = 15;
    Instancia I = generar(N);

    vector<double> nn(N), ci(N);
    int gCI = 0, gNN = 0, emp = 0;
    printf("%-7s %10s %12s %8s\n", "Origen", "NN", "Ins. barata", "Gana");
    for (int o = 0; o < N; o++) {
        nn[o] = costo(vecinoCercano(o, I.d), I.d);
        ci[o] = costo(insercionBarata(o, I.d), I.d);
        bool empate = fabs(nn[o] - ci[o]) < EPS;
        (empate ? emp : ci[o] < nn[o] ? gCI : gNN)++;
        printf("%-7d %10.3f %12.3f %8s\n", o, nn[o], ci[o], empate ? "empate" : ci[o] <
        ↪  nn[o] ? "CI" : "NN");
    }

    Resumen a = resumir(nn), b = resumir(ci);
    double tNN = tiempoMs(vecinoCercano, I.d), tCI = tiempoMs(insercionBarata, I.d);
    printf("\n%-12s %9s %9s %9s %12s %10s %11s\n", "", "Mejor", "Peor", "Media", "Casos
    ↪  mejor", "Distintos", "Tiempo(ms)");
    printf("%-12s %9.3f %9.3f %9.3f %9d/%d %10d %11.4f\n", "NN", a.mejor, a.peor, a.media,
    ↪  a.enMejor, N, a.distintos, tNN);
    printf("%-12s %9.3f %9.3f %9.3f %9d/%d %10d %11.4f\n", "Ins. barata", b.mejor, b.peor,
    ↪  b.media, b.enMejor, N, b.distintos, tCI);

    printf("\nPor origen: CI gana %d, NN gana %d, empates %d\n", gCI, gNN, emp);
    printf("Mejora media de CI sobre NN: %.2f%% | Mejor global: %s\n", 100 * (a.media -
    ↪  b.media) / a.media,
           fabs(a.mejor - b.mejor) < EPS ? "empate" : a.mejor < b.mejor ? "NN" : "CI");
    printf("Costo de tiempo: CI es %.1f veces más lenta que NN\n", tCI / tNN);
}
```

- /relleno

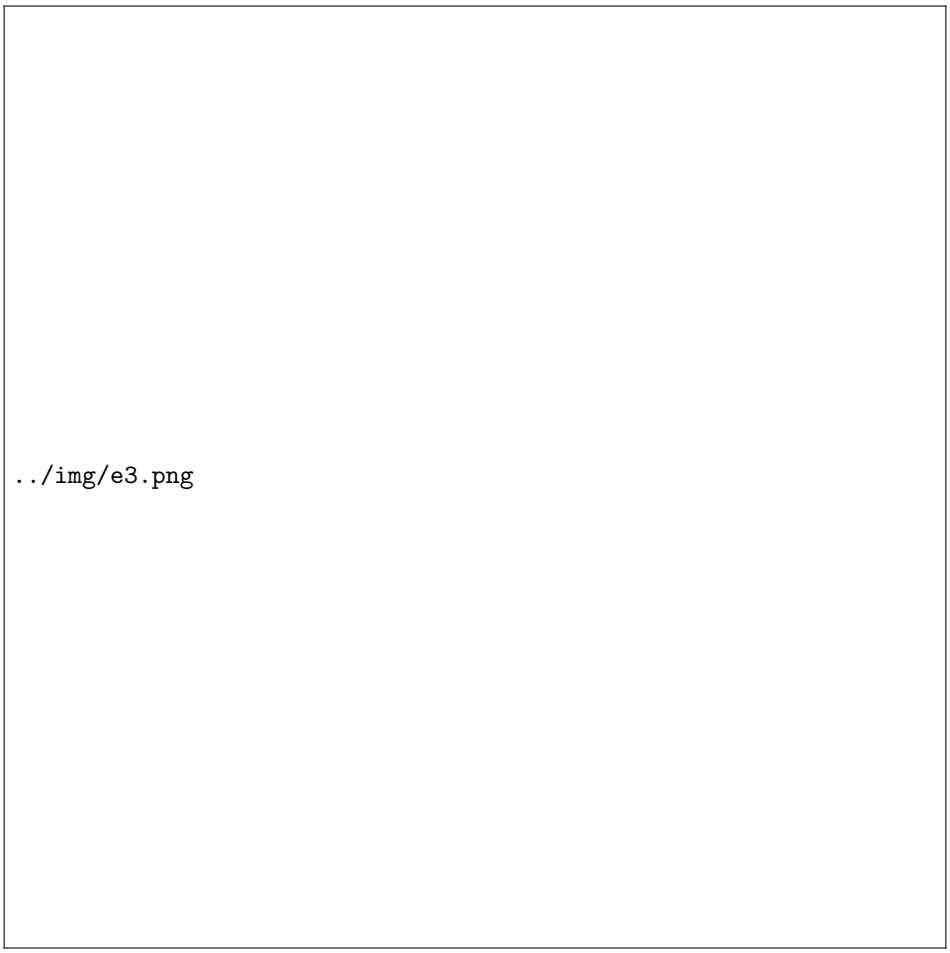


Figura 5: Ejecución del ejercicio 3

2.4. Ejercicio 4

Mide el tiempo de ejecución de Nearest Neighbor + 2-opt para instancias de 20, 50 y 100 ciudades, y compáralo contra el tiempo que tomaría fuerza bruta para esos mismos tamaños (estimado, sin ejecutarlo).

```cpp
// Ejercicio 4: tiempo de Nearest Neighbor + 2-opt para 20, 50 y 100 ciudades,
// contra el tiempo ESTIMADO (no ejecutado) de la fuerza bruta: (n-1)!/2 rutas.
#include "tsp.h"

// Tiempo medio (ms) de una corrida de NN + 2-opt desde la ciudad 0 (solo el algoritmo: sin generar datos ni imprimir).
// Se repite porque una sola corrida es muy corta; devuelve tambin el costo de la ruta obtenida.
double tiempoHeuristica(const Mat& d, int reps, double& costoFinal) {
    auto t0 = chrono::steady_clock::now();
    for (int k = 0; k < reps; k++) costoFinal = costo(dosOpt(vecinoCercano(0, d), d), d);
    return chrono::duration<double, milli>(chrono::steady_clock::now() - t0).count() / reps;
}

```

```cpp
// Segundos que tarda la fuerza bruta en procesar UNA ruta: se mide enumerando permutaciones reales
// (next_permutation + costo) y se extrapola al total (n-1)!/2, sin ejecutar la enumeracin completa.
double segundosPorRuta(const Mat& d, long muestra = 1000000) {
    int n = d.size();
    Ruta r(n);
    iota(r.begin(), r.end(), 0);  // la ciudad 0 queda fija: ah se evitan las n rotaciones del mismo ciclo
    double sumidero = 0;
    auto t0 = chrono::steady_clock::now();
    for (long k = 0; k < muestra; k++) {
        sumidero += costo(r, d);
        next_permutation(r.begin() + 1, r.end());
    }
    double s = chrono::duration<double>(chrono::steady_clock::now() - t0).count();
    return s / muestra + 0 * sumidero;  // sumidero evita que el compilador elimine el bucle
}

int main() {
    const double SEG_ANIO = 365.25 * 24 * 3600, EDAD_UNIVERSO = 1.38e10;  // años
    printf("%-5s %12s %14s %14s %14s %14s %12s\n", "n", "Costo", "NN+2opt (ms)", "s/ruta (BF)", "Rutas (n-1)!/2", "BF (aos)", "BF/NN+2opt");
    for (int n : {20, 50, 100}) {
        Instancia I = generar(n);  // misma semilla: las primeras ciudades coinciden con las de los otros ejercicios
        double c = 0, msH = tiempoHeuristica(I.d, 20000 / n, c);
        double spr = segundosPorRuta(I.d);
        double rutas = exp(lgamma(n)) / 2;  // lgamma(n) = ln((n-1)!); evita desbordar un entero
        double segBF = rutas * spr;
        printf("%-5d %12.3f %14.4f %14.2e %14.2e %14.2e %12.2e\n", n, c, msH, spr, rutas, segBF / SEG_ANIO, segBF / (msH / 1000));
    }
    printf("\nPara comparar: la edad del universo es ~%.2e años.\n", EDAD_UNIVERSO);
    printf("(BF = fuerza bruta, estimada: tiempo medido por ruta x nmero de rutas; no se ejecut.)\n");
}
```

- /relleno

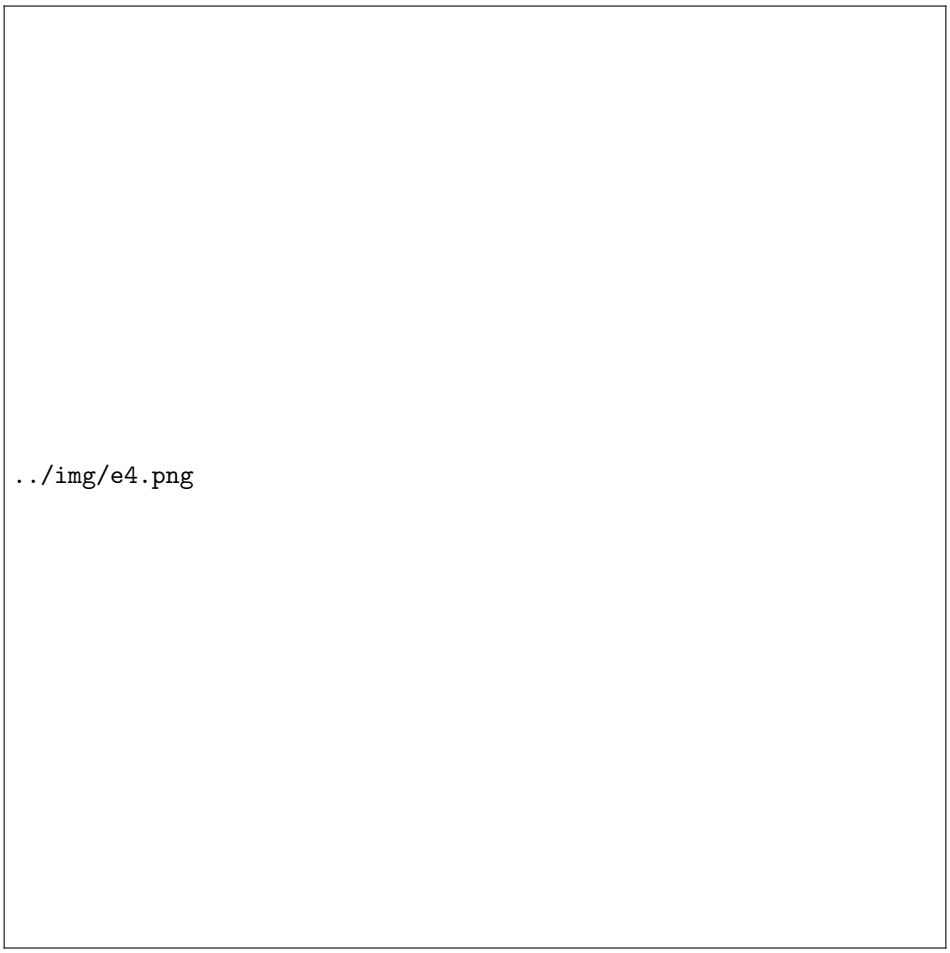


Figura 6: Ejecución del ejercicio 4

2.5. Ejercicio 5

Aplica Nearest Neighbor + 2-opt a un caso real: una lista de direcciones o puntos de interés de tu ciudad (puedes usar coordenadas aproximadas), y calcula la ruta de reparto o visita más corta posible.

```
1
```

- /relleno

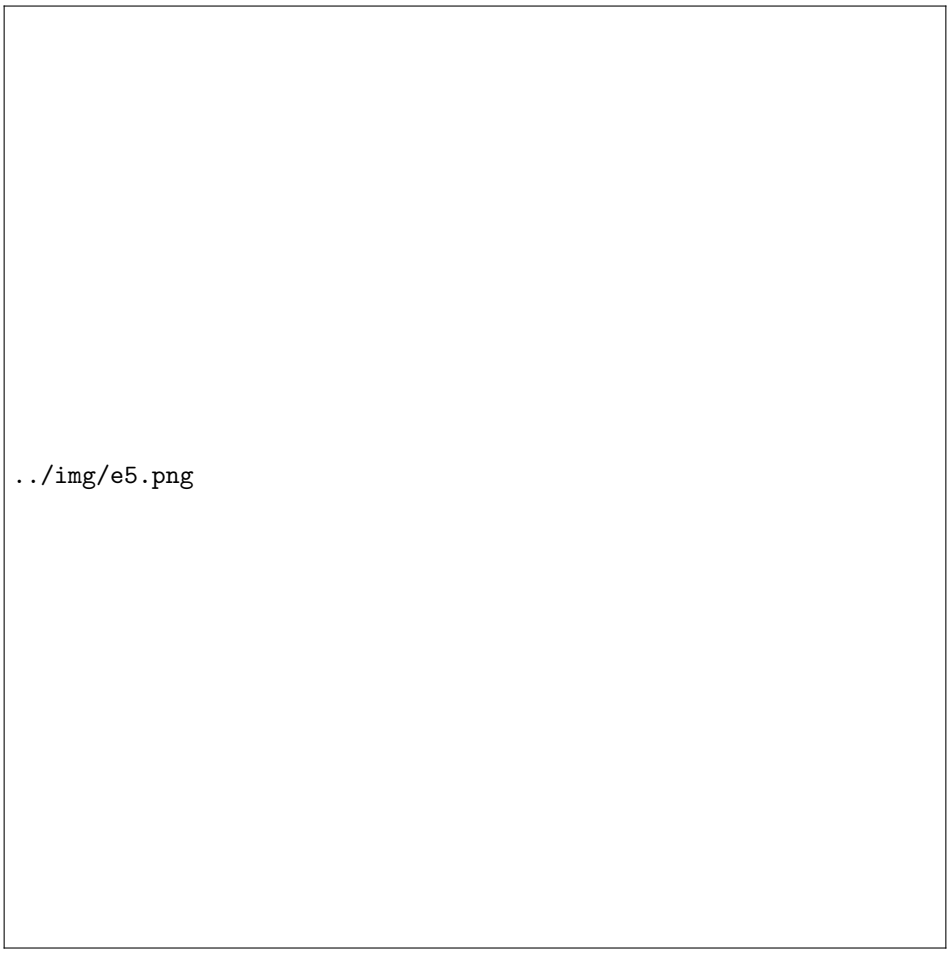


Figura 7: Ejecución del ejercicio 5

3. Cuestionario

- ¿Cuál es la diferencia entre un método aproximado y un método heurístico?
- ¿Por qué Nearest Neighbor puede terminar con un salto muy costoso al final de la ruta? Usa como ejemplo el inicio en la ciudad 2 de la Tabla 3.
- ¿Qué significa que 2-opt encuentre un "óptimo local¿ ¿Por qué no garantiza el óptimo global?
- ¿Por qué combinar una heurística constructiva con una de búsqueda local suele dar mejores resultados que usar cualquiera de las dos por separado?
- Comparando la Figura 5 (y la Tabla 5) de este laboratorio con lo visto en el Laboratorio 8, ¿por qué Nearest Neighbor + 2-opt es preferible a Monte Carlo incluso con un presupuesto de cómputo similar?

4. Conclusiones